IDSS 2019-2020 Examination Solutions

University of Glasgow Introduction to Data Science and Systems (M) Date: Thursday 19 December 2019 Duration: 1 hour 30 minutes Total Marks: 60

Question 1: Linear Algebra, Probability, Visualisation and Optimisation (20 marks)

Part (a): Convert coupled equations to matrix form Ab = c [3 marks]

Given equations:

```
−14 + xα + zγ = −yβ
2xα − yzβ + 8 = −xγ + xα
−zγ = −5 − yβ
```

Given values: x = 1, y = 2, z = 3

Step 1: Simplify each equation

Equation 1: −14 + xα + zγ = −yβ - Rearranging: xα + yβ + zγ = 14 - Substituting values: 1·α + 2·β + 3·γ = 14 - **α + 2β + 3γ = 14**

Equation 2: 2xα − yzβ + 8 = −xγ + xα - Simplifying: 2xα − xα − yzβ + xγ = −8 - xα − yzβ + xγ = −8 - Substituting values: 1·α − (2)(3)·β + 1·γ = −8 - **α − 6β + γ = −8**

Equation 3: −zγ = −5 − yβ - Rearranging: yβ + zγ = 5 - Substituting values: 2·β + 3·γ = 5 - **0·α + 2β + 3γ = 5**

Step 2: Matrix form Ab = c

```
A = | 1   2   3 |      b = | α |      c = | 14 |
    | 1  -6   1 |          | β |          | -8 |
    | 0   2   3 |          | γ |          |  5 |
```

Answer:

```
| 1   2   3 | | α |   | 14 |
| 1  -6   1 | | β | = | -8 |
| 0   2   3 | | γ |   |  5 |
```

Part (b)(i): Define optimization problem without matrix inversion [2 marks]

Optimization Problem Formulation:

We want to solve **Ab = c** for **b** without using matrix inversion.

Objective Function:

Minimize the squared error (loss function):

$$L(b) = ||Ab - c||^2 = (Ab - c)^T(Ab - c)$$

Or equivalently:

$$L(b) = \Sigma_i (\Sigma_j A_{ij} b_j - c_i)^2$$

This is a **least squares optimization problem**.

Gradient:

The partial derivative with respect to parameter b_k is:

$$\partial L/\partial b_k = 2\Sigma_i A_{ik}(\Sigma_j A_{ij} b_j - c_i)$$

Or in matrix form:

$$\nabla L(b) = 2A^T(Ab - c)$$

This gradient can be computed using only matrix-vector multiplications and transposes, without requiring matrix inversion.

Part (b)(ii): Gradient descent update equations and convergence conditions [4 marks]

Gradient Descent Update Equation:

The standard gradient descent update rule is:

$b^{(t+1)} = b^{(t)} - \eta \nabla L(b^{(t)})$

Where: - $b^{(t)}$ is the parameter vector at iteration t - η is the learning rate (step size) - $\nabla L(b^{(t)})$ is the gradient at iteration t

Substituting our gradient:

$b^{(t+1)} = b^{(t)} - 2\eta A^T(Ab^{(t)} - c)$

Expanded form:

$b^{(t+1)} = b^{(t)} - 2\eta A^T A b^{(t)} + 2\eta A^T c$

Component-wise update:

For each parameter b_k:

$b_k^{(t+1)} = b_k^{(t)} - 2\eta \sum_i A_{ik}(\sum_j A_{ij} b_j^{(t)} - c_i)$

Convergence Conditions:

The gradient descent algorithm is **guaranteed to converge** under the following conditions:

1. **Learning Rate Constraint:**
2. The learning rate η must be chosen such that: **$0 < \eta < 1/\lambda_{max}$**
3. where λ_{max} is the largest eigenvalue of the matrix A^TA
4. This ensures the algorithm doesn't overshoot the minimum
5. **Convexity:**
6. The objective function $L(b) = ||Ab - c||^2$ is **convex** (it's a quadratic function)
7. This guarantees a unique global minimum and no local minima
8. **Lipschitz Continuity:**
9. The gradient must be Lipschitz continuous
10. For our problem, $\nabla L(b) = 2A^T(Ab - c)$ is Lipschitz continuous with constant $L = 2||A^TA||$
11. **Sufficient Iterations:**
12. The algorithm must run for sufficient iterations
13. Convergence rate: O(1/t) for smooth convex functions

Practical Considerations:

- **Convergence criterion:** Stop when $||\nabla L(b^{(t)})|| < \varepsilon$ or $||b^{(t+1)} - b^{(t)}|| < \varepsilon$

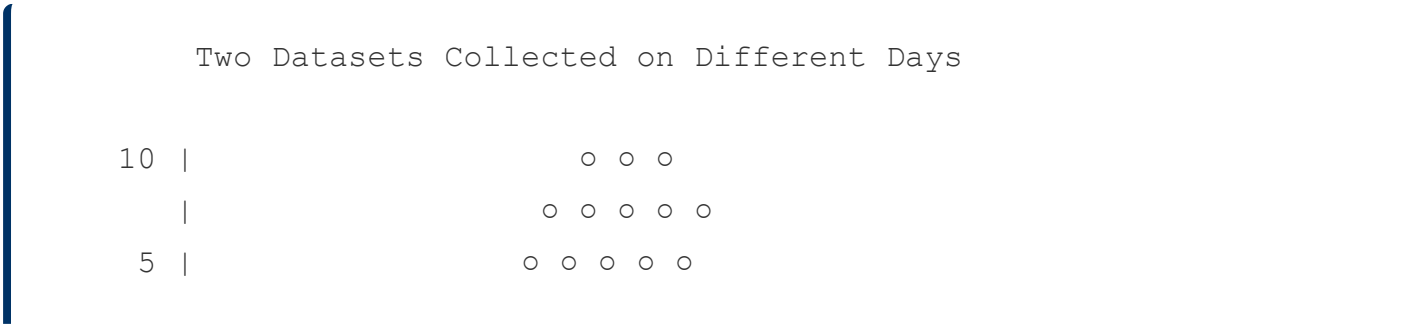
- **Learning rate selection:** Common choice is $\eta = 1/(2\lambda_{max}(A^TA))$
- **Alternative:** Use adaptive learning rates (e.g., Adam, RMSprop)

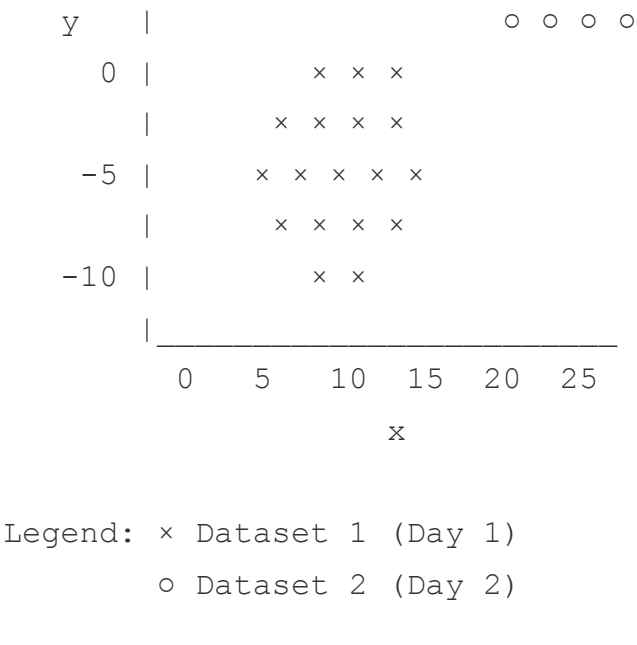
Part (c)(i): Criticise Figure 1 and redraw [3 marks]

Criticisms of Figure 1:

1. **Aspect Ratio Problem:**
2. The x-axis ranges from 0 to 27, while y-axis ranges from -10 to 10
3. Unequal scaling distorts the true geometric relationship between clusters
4. Makes it difficult to accurately assess cluster separation and shape
5. **Missing Axis Labels:**
6. No units or description for what x and y represent
7. Makes the visualization less informative
8. **No Title:**
9. The figure lacks a descriptive title
10. Context about what data is being shown is missing
11. **Poor Color/Symbol Differentiation:**
12. Both clusters use the same 'x' marker
13. No clear visual distinction between the two datasets
14. Should use different colors or shapes for each cluster
15. **Equal Aspect Ratio Not Enforced:**
16. For spatial data, equal aspect ratio is important to preserve distances
17. Current scaling makes one cluster appear more elongated than it actually is

Improved Sketch:





Key Improvements: - Equal aspect ratio (1:1) - Different symbols for different datasets (× and ○) - Clear axis labels with units - Descriptive title - Legend to identify datasets - Grid lines for easier reading (optional)

Part (c)(ii): Parameterise Normal distributions for (x,y) datasets [3 marks]

Multivariate Normal Distribution:

For 2D data (x, y), we use a **bivariate normal distribution**:

N(μ, Σ)

where: - **μ** is the mean vector (2D) - **Σ** is the covariance matrix (2×2)

Parameters for Each Dataset:

Dataset 1 (Day 1): - $\boldsymbol{\mu}_1 = [\mu\_x1, \mu\_y1]^T$ - mean vector - Shape: (2, 1) or (2,) - μ_x1 = mean of x values in dataset 1 - μ_y1 = mean of y values in dataset 1

- $\boldsymbol{\Sigma}_1$ - covariance matrix
- Shape: (2, 2) $\Sigma_1 = \begin{vmatrix} \sigma^2\_x1 & \text{cov}(x,y)_1 \\ \text{cov}(x,y)_1 & \sigma^2\_y1 \end{vmatrix}$ where:
- σ^2_x1 = variance of x in dataset 1
- σ^2_y1 = variance of y in dataset 1
- $\text{cov}(x,y)_1$ = covariance between x and y in dataset 1

Dataset 2 (Day 2): - $\boldsymbol{\mu}_2 = [\mu\_x2, \mu\_y2]^T$ - mean vector - Shape: (2, 1) or (2,)

- $\boldsymbol{\Sigma}_2$ - covariance matrix
- Shape: (2, 2) $\Sigma_2 = \begin{vmatrix} \sigma^2\_x2 & \text{cov}(x,y)_2 \\ \text{cov}(x,y)_2 & \sigma^2\_y2 \end{vmatrix}$

Estimation Formulas:

For dataset k with N_k observations:

Mean:

$$\mu_k = (1/N_k) \Sigma_i [x_i, y_i]^T$$

Covariance Matrix:

$$\Sigma_k = (1/(N_k-1)) \Sigma_i (p_i - \mu_k)(p_i - \mu_k)^T$$

where $p_i = [x_i, y_i]^T$

Array Shapes Summary: - μ_1, μ_2: shape (2,) - two elements each - Σ_1, Σ_2: shape (2, 2) - symmetric matrices

Probability Density Function:

$$p(x, y \mid \mu, \Sigma) = (1/(2\pi|\Sigma|^(1/2))) \exp(-1/2 (p - \mu)^T\Sigma^{-1}(p - \mu))$$

Part (c)(iii): Eigendecomposition for major axis of variation [5 marks]

Theory:

Eigendecomposition of the covariance matrix reveals the principal directions of variation in the data.

For covariance matrix Σ:

$$\Sigma v = \lambda v$$

where: - **v** is an eigenvector (direction of variation) - **λ** is an eigenvalue (magnitude of variation in that direction)

Eigendecomposition:

$$\Sigma = V\Lambda V^T$$

where: - $\mathbf{V = [v_1, v_2]}$ is the matrix of eigenvectors - $\mathbf{\Lambda = diag(\lambda_1, \lambda_2)}$ is the diagonal matrix of eigenvalues - Typically ordered: $\lambda_1 \geq \lambda_2$

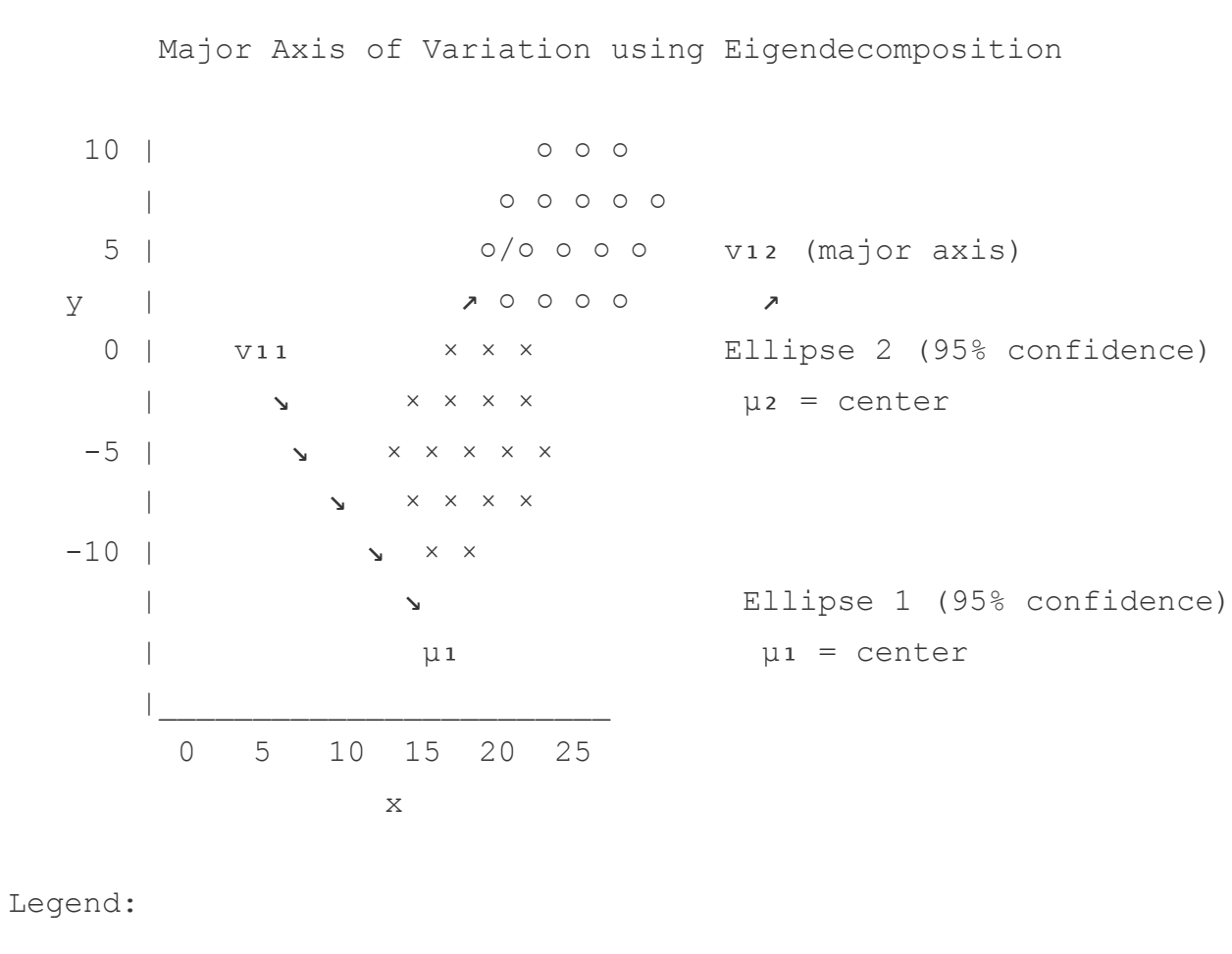
Physical Interpretation:

1. **Major axis:** Direction of eigenvector v_1 (largest eigenvalue λ_1)
2. Direction of maximum variance
3. Length proportional to $\sqrt{\lambda_1}$
4. **Minor axis:** Direction of eigenvector v_2 (smallest eigenvalue λ_2)
5. Direction of minimum variance
6. Length proportional to $\sqrt{\lambda_2}$
7. **Eigenvectors are orthogonal:** $v_1 \perp v_2$

Process:

For each dataset k: 1. Compute covariance matrix Σ_k 2. Find eigenvalues: λ_{1k}, λ_{2k} 3. Find corresponding eigenvectors: v_{1k}, v_{2k} 4. Major axis is along v_{1k} (eigenvector with larger eigenvalue)

Sketch:



```
× Dataset 1 points
○ Dataset 2 points
μ1, μ2 = mean vectors (centers)
Ellipses = 95% confidence regions
v11 = major eigenvector for dataset 1 (pointing down-right)
v12 = major eigenvector for dataset 2 (pointing up-right)
Eigenvectors show principal directions of variation
```

Key Features in Sketch:

1. **Data points:** Scatter plots for both datasets
2. **Mean vectors μ_1, μ_2:** Centers of each distribution (marked)
3. **Confidence ellipses:**
4. Semi-major axis = $k\sqrt{\lambda_1}$ along v_1
5. Semi-minor axis = $k\sqrt{\lambda_2}$ along v_2
6. where $k \approx 2.45$ for 95% confidence in 2D
7. **Eigenvectors:**
8. v_{1k}: Major axis (longer arrow)
9. v_{2k}: Minor axis (shorter arrow, perpendicular to v_{1k})
10. Both originate from mean μ_k
11. **Observations from sketch:**
12. Dataset 1: Negative correlation (down-right major axis)
13. Dataset 2: Positive correlation (up-right major axis)
14. Different orientations show different correlation structures

Mathematical Details:

For 95% confidence ellipse boundary:

$$(p - \mu)^T \Sigma^{-1} (p - \mu) = \chi^2_{0.05,2} \approx 5.99$$

Or in terms of eigenvectors:

$$p = \mu + \sqrt{\lambda_1} \cdot \cos(\theta) \cdot v_1 + \sqrt{\lambda_2} \cdot \sin(\theta) \cdot v_2$$

for $\theta \in [0, 2\pi]$

Question 2: Text Processing in Data Science (20 marks)

Part (a)(i): Calculate cosine similarity [3 marks]

Given: - D1 = [4, 2, 0] - D2 = [2, 0, 4]

Cosine Similarity Formula:

$$\cos(\theta) = (D1 \cdot D2) / (||D1|| \cdot ||D2||)$$

where: - **D1 · D2** is the dot product - **||D1||** is the Euclidean norm (magnitude) of D1 - **||D2||** is the Euclidean norm of D2

Step 1: Calculate Dot Product

$$\begin{aligned} D1 \cdot D2 &= \Sigma_i \, D1_i \times D2_i \\ &= (4 \times 2) + (2 \times 0) + (0 \times 4) \\ &= 8 + 0 + 0 \\ &= 8 \end{aligned}$$

Step 2: Calculate Magnitudes

$$\begin{aligned} ||D1|| &= \sqrt{(\Sigma_i \, D1_i^2)} \\ &= \sqrt{(4^2 + 2^2 + 0^2)} \\ &= \sqrt{(16 + 4 + 0)} \\ &= \sqrt{20} \\ &= 2\sqrt{5} \end{aligned}$$

$$\begin{aligned} ||D2|| &= \sqrt{(\Sigma_i \, D2_i^2)} \\ &= \sqrt{(2^2 + 0^2 + 4^2)} \\ &= \sqrt{(4 + 0 + 16)} \\ &= \sqrt{20} \\ &= 2\sqrt{5} \end{aligned}$$

Step 3: Calculate Cosine Similarity

```
cos(θ) = 8 / (2√5 · 2√5)
       = 8 / (4 × 5)
       = 8 / 20
       = 2/5
       = 0.4
```

Answer: The cosine similarity between D1 and D2 is 0.4 (or 2/5)

Interpretation: - Cosine similarity ranges from -1 to 1 - 0.4 indicates moderate similarity - The angle θ = arccos(0.4) ≈ 66.4°

Part (a)(ii): Application of cosine similarity [2 marks]

Application: Document Similarity / Information Retrieval

Description:

Cosine similarity is widely used in **document search engines** and **recommendation systems**. When a user enters a search query, the system: 1. Converts the query into a term frequency vector 2. Computes cosine similarity between query vector and all document vectors 3. Ranks documents by similarity score 4. Returns most similar documents

Why Cosine Similarity?

Key Geometric Properties:

1. **Length Invariance:**
2. Cosine similarity measures the **angle** between vectors, not magnitude
3. Documents of different lengths can be compared fairly
4. A short document with relevant terms can rank highly against long documents

5. Formula: cos(θ) = (A·B)/(||A||·||B||) - normalizes by magnitude

6. **Focus on Direction:**

7. Emphasizes the **proportion** of terms rather than absolute counts
8. Two documents with same term proportions have similarity = 1

9. Example: [1, 1, 0] and [10, 10, 0] have cosine similarity = 1

10. **Bounded Range [-1, 1]:**

11. For term frequency (non-negative values), range is [0, 1]

12. Easy to interpret: 1 = identical direction, 0 = orthogonal (no similarity)

13. **Sparse Vector Efficiency:**

14. Only non-zero terms contribute to dot product
15. Efficient computation for high-dimensional sparse text vectors

Importance for Application:

- **Document length normalization:** Long documents don't automatically dominate
- **Scale invariance:** "cat" appearing 2 times vs 20 times treated proportionally
- **Interpretability:** Clear geometric meaning - angular distance
- **Computational efficiency:** Simple dot product operations

Alternative Example Application: User-item recommendation systems (Netflix, Amazon) where user preferences are compared using cosine similarity of rating vectors.

Part (a)(iii): Python code for TF-IDF cosine similarity [3 marks]

Given:

```
docs = ('The sky is green',
        'The sun is yellow',
        'We can see the shining sun, the bright sun in the sky')
```

Python Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Given documents
docs = ('The sky is green',
        'The sun is yellow',
        'We can see the shining sun, the bright sun in the sky')

# Step 1: Create TF-IDF vectorizer
vectorizer = TfidfVectorizer()

# Step 2: Fit and transform documents to TF-IDF matrix
tfidf_matrix = vectorizer.fit_transform(docs)
```

```
# Step 3: Compute pairwise cosine similarity matrix
similarity_matrix = cosine_similarity(tfidf_matrix)

# Step 4: Display results
print("TF-IDF Feature Names:")
print(vectorizer.get_feature_names_out())
print("\nTF-IDF Matrix:")
print(tfidf_matrix.toarray())
print("\nCosine Similarity Matrix:")
print(similarity_matrix)
```

Expected Output Structure:

```
TF-IDF Feature Names:
['bright' 'can' 'green' 'in' 'is' 'see' 'shining' 'sky' 'sun' 'the' '

TF-IDF Matrix:
[[0.    0.    0.71  0.    0.41  0.    0.    0.41  0.    0.41  0.    0
 [0.    0.    0.    0.    0.41  0.    0.    0.    0.71  0.41  0.    0
 [0.27  0.27  0.    0.27  0.    0.27  0.27  0.21  0.43  0.42  0.27  0

Cosine Similarity Matrix:
[[1.    0.17  0.17]
 [0.17  1.    0.49]
 [0.17  0.49  1.  ]]
```

Explanation:

1. **TfidfVectorizer:** Converts documents to TF-IDF weighted vectors
2. TF (Term Frequency): How often term appears in document
3. IDF (Inverse Document Frequency): How rare term is across all documents
4. TF-IDF = TF × IDF (down-weights common words)
5. **fit_transform:** Creates vocabulary and transforms documents in one step
6. **cosine_similarity:** Computes pairwise similarity between all document pairs
7. Diagonal = 1 (document similar to itself)

8. Symmetric matrix

9. **Similarity matrix interpretation:**

10. similarity_matrix[i][j] = similarity between document i and document j
11. Doc 0 and 1: low similarity (0.17) - different content
12. Doc 1 and 2: moderate similarity (0.49) - both mention "sun"

Alternative with manual calculation:

```
# Alternative: Access specific similarities
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

docs = ('The sky is green',
        'The sun is yellow',
        'We can see the shining sun, the bright sun in the sky')

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(docs)

# Compute full similarity matrix
similarity_matrix = cosine_similarity(tfidf_matrix, tfidf_matrix)

# Or compute specific pair
sim_0_1 = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix[1:2])[0][
```

Part (a)(iv): Lemmatization vs Stemming [2 marks]

Lemmatization:

Definition: Lemmatization is the process of reducing words to their **base or dictionary form** (called the **lemma**) using vocabulary and morphological analysis.

Process: - Uses language dictionary and grammatical rules - Considers the context and part of speech (POS) - Returns valid words that exist in the language

Examples:

```
"running" → "run" (verb)
"ran" → "run" (verb)
"better" → "good" (adjective)
"geese" → "goose" (noun)
"is", "are", "was" → "be" (verb)
```

Characteristics: - **Accurate:** Produces linguistically correct base forms - **Context-aware:** Same word can have different lemmas based on POS - "meeting" (verb) → "meet" - "meeting" (noun) → "meeting" - **Slower:** Requires dictionary lookup and POS analysis - **Meaningful:** Output is always a real word

---

Stemming:

Definition: Stemming is the process of reducing words to their **root form** (called the **stem**) by removing suffixes using heuristic rules.

Process: - Applies simple rule-based transformations - Chops off word endings without linguistic analysis - Doesn't require dictionary or context

Examples (Porter Stemmer):

```
"running" → "run"
"runs" → "run"
"fairly" → "fair"
"trouble" → "troubl" (not a real word!)
"connection" → "connect"
"connected" → "connect"
```

Characteristics: - **Fast:** Simple rule-based operations - **Crude:** May produce non-words ("troubl", "univers") - **Over-stemming:** Different words may have same stem - "wander" → "wand", "wander" → "wand" (correct) - "universe" → "univers", "university" → "univers" (incorrect - different meanings!) - **Under-stemming:** Similar words may have different stems

---

Comparison:

Aspect	Stemming	Lemmatization
Output	Stem (may not be real word)	Lemma (always real word)
Method	Rule-based suffix removal	Dictionary + morphological analysis
Speed	Fast	Slower
Accuracy	Lower	Higher
Context	No context consideration	Uses POS tags
Example	"studies" → "studi"	"studies" → "study"
Use case	Quick preprocessing, search engines	NLU, semantic analysis

When to use:

- **Stemming:** When speed is critical, exact words less important (e.g., search engines, basic text classification)
- **Lemmatization:** When semantic accuracy matters (e.g., sentiment analysis, machine translation, question answering)

Common Algorithms:

- **Stemming:** Porter Stemmer, Snowball Stemmer, Lancaster Stemmer
- **Lemmatization:** WordNet Lemmatizer (NLTK), SpaCy Lemmatizer

---

Part (b)(i): K-means algorithm and properties [3 marks]

K-means Clustering Algorithm:

Pseudo-code:

```
Algorithm: K-means Clustering
Input:
  - Dataset D = {x1, x2, ..., xn} (n data points)
  - Number of clusters k
  - Maximum iterations max_iter
  - Convergence threshold ε
```

```
Output:
  - Cluster centroids C = {c₁, c₂, ..., cₖ}
  - Cluster assignments A = {a₁, a₂, ..., aₙ} where aᵢ ∈ {1,...,k}

1. INITIALIZATION:
   Randomly select k data points as initial centroids c₁, c₂, ..., cₖ
   (or use k-means++ for better initialization)

2. REPEAT until convergence (or max_iter reached):

   a) ASSIGNMENT STEP:
      For each data point xᵢ in D:
          aᵢ = argmin_j ||xᵢ - cⱼ||²    (assign to nearest centroid)

      // Assign each point to closest cluster based on Euclidean dist

   b) UPDATE STEP:
      For each cluster j from 1 to k:
          cⱼ = mean of all points assigned to cluster j
          cⱼ = (1/|Cⱼ|) Σ_{xᵢ ∈ Cⱼ} xᵢ

      // Recompute centroid as mean of assigned points

   c) CHECK CONVERGENCE:
      If ||cⱼ_new - cⱼ_old||² < ε for all j:
          BREAK
      Or if no assignments changed:
          BREAK

3. RETURN centroids C and assignments A
```

Detailed Description:

1. **Initialization:** Choose k random points as initial cluster centers
2. **Assignment:** Assign each data point to nearest centroid using distance metric (typically Euclidean)
3. **Update:** Recalculate centroids as mean of all points in each cluster

4. **Iterate:** Repeat steps 2-3 until convergence (centroids don't change or change < threshold)

Three Key Clustering Properties of K-means:

1. Convex Clusters (Spherical/Globular Clusters)

- **Property:** K-means produces clusters that are convex and roughly spherical
- **Reason:** Uses Euclidean distance and mean as centroid
- **Implication:**
- Works well for globular, equally-sized clusters
- Struggles with non-convex shapes (crescents, rings, elongated clusters)
- Assumes clusters have similar variance
- **Example:** K-means fails on two interleaved crescent shapes

2. Hard Assignment (Crisp Clustering)

- **Property:** Each data point belongs to exactly ONE cluster
- **Characteristics:**
- Binary membership: point i either in cluster j or not
- No probabilistic/fuzzy membership
- Clear decision boundaries (Voronoi tessellation)
- **Comparison:** Unlike soft clustering (e.g., Gaussian Mixture Models) where points can partially belong to multiple clusters
- **Implication:** Points on cluster boundaries are forced into one cluster

3. Minimizes Within-Cluster Sum of Squares (WCSS)

- **Property:** K-means optimizes the following objective function:

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} ||x_i - c_j||^2$$

where C_j is cluster j and c_j is its centroid

- **Characteristics:**
- Also called "inertia" or "within-cluster variance"
- Measures compactness of clusters
- Guaranteed to decrease (or stay same) with each iteration
- **Convergence:**
- Converges to local minimum (not guaranteed global)
- Different initializations may give different results
- K-means++ initialization helps find better local optima

- **Limitation:**
- Favors clusters of similar sizes
- Sensitive to outliers (squared distance penalizes outliers heavily)

Part (b)(ii): K-means application in social media advertising [3 marks]

Task: User Segmentation for Targeted Advertising

Objective: Cluster users of a social media platform based on their interests and behavior to deliver personalized advertisements.

Implementation Details:

1. Textual Features:

Extract features from user-generated content:

a) **User Profile Text:** - Bio/description - Posted content (tweets, status updates, comments) - Liked/shared content

b) **Specific Features:** - **Interest keywords:** Technology, sports, fashion, food, travel, etc. - **Hashtags used:** #AI, #MachineLearning, #Fashion, etc. - **Engagement patterns:** Types of ads clicked, posts liked - **Demographic text:** Location mentions, job titles

2. Feature Representation:

TF-IDF Vectors:

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Aggregate all user text
user_texts = [user1_posts + user1_bio + user1_comments,
              user2_posts + user2_bio + user2_comments,
              ...]

# Create TF-IDF representation
vectorizer = TfidfVectorizer(
    max_features=10000,      # Limit vocabulary size
    min_df=5,                # Ignore rare terms
    max_df=0.7,              # Ignore too common terms
```

```
    stop_words='english',     # Remove stop words
    ngram_range=(1, 2)        # Use unigrams and bigrams
)

feature_matrix = vectorizer.fit_transform(user_texts)
# Result: Sparse matrix of shape (n_users, 10000)
```

Alternative Representations: - **Word embeddings:** Average Word2Vec or Doc2Vec vectors (dense, semantic) - **Topic models:** LDA topic distributions as features - **Behavioral features:** Click-through rates, time spent on categories

3. Similarity Function:

Cosine Similarity:

```
similarity(u₁, u₂) = (u₁ · u₂) / (||u₁|| · ||u₂||)
```

Why cosine? - TF-IDF vectors are sparse and high-dimensional - Length invariance: Active users (many posts) vs passive users compared fairly - Focuses on interest overlap, not volume of activity - Standard in text clustering applications

Distance metric for k-means:

```
distance(u₁, u₂) = 1 - cosine_similarity(u₁, u₂)
```

Or use Euclidean distance on normalized TF-IDF vectors (equivalent to cosine).

4. Addressing Scale Issues:

Problem: Millions/billions of users and high-dimensional features

Solutions:

a) **Mini-batch K-means:** ```python from sklearn.cluster import MiniBatchKMeans

kmeans = MiniBatchKMeans(n_clusters=100, batch_size=10000, # Process 10K users at a time max_iter=100) kmeans.fit(feature_matrix) ``` - Processes random samples (batches) instead of full dataset - Much faster, slightly less accurate - Suitable for online learning

b) **Dimensionality Reduction:** ```python from sklearn.decomposition import TruncatedSVD

svd = TruncatedSVD(n_components=100) reduced_features = svd.fit_transform(feature_matrix) ``` - Reduce from 10,000 to 100 dimensions - Latent Semantic Analysis (LSA) for text - Faster

computation, removes noise

c) **Sampling:** - Cluster representative sample (e.g., 10% of users) - Assign remaining users to nearest cluster

d) **Distributed Computing:** - Use Spark MLlib for distributed k-means - Parallel processing across multiple machines

e) **Hierarchical Approach:** - First cluster into broad categories (k=20) - Then sub-cluster within each category - Creates hierarchical user taxonomy

5. Implementation Pipeline:

```
from sklearn.cluster import MiniBatchKMeans
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize

# 1. Feature extraction
vectorizer = TfidfVectorizer(max_features=5000)
features = vectorizer.fit_transform(user_texts)

# 2. Normalization (for cosine distance)
features_normalized = normalize(features, norm='l2')

# 3. Clustering
k = 50  # 50 user segments
kmeans = MiniBatchKMeans(n_clusters=k, batch_size=10000)
user_clusters = kmeans.fit_predict(features_normalized)

# 4. Assign ads to clusters
# Each cluster gets tailored advertisements
for cluster_id in range(k):
    cluster_users = users[user_clusters == cluster_id]
    cluster_keywords = get_top_terms(kmeans.cluster_centers_[cluster_
    assign_ads(cluster_id, cluster_keywords)
```

6. Outcome: - **50 user segments** with distinct interest profiles - Each segment receives targeted ads matching their interests - Example clusters: "Tech Enthusiasts", "Fashion Lovers", "Sports Fans", etc.

Part (b)(iii): Speeding up k-means for large datasets [2 marks]

Problem Analysis:

K-means is slow on very large datasets because:

1. **Computational Complexity:**
2. **Assignment step:** O(n × k × d) per iteration
    - n = number of data points
    - k = number of clusters
    - d = number of features/dimensions
3. Must compute distance from each of n points to each of k centroids
4. **Large-Scale Issues:**
5. With millions of data points, each iteration is expensive
6. Many iterations needed for convergence
7. All data must fit in memory
8. **Fixed Constraints:**
9. Number of clusters k is fixed (can't reduce)
10. Number of features d is fixed (can't reduce)
11. Only n (data size) and iterations can be optimized

Solution: Mini-batch K-means

Modification:

Instead of using ALL n data points in each iteration, use a random **mini-batch** (subset) of size b << n.

Modified Algorithm:

```
Mini-batch K-means:

1. Initialize k centroids (same as standard k-means)

2. For t = 1 to max_iterations:

   a) Sample random mini-batch M of size b from dataset
      (typically b = 100 to 10,000)
```

```
   b) ASSIGNMENT (only for mini-batch):
      For each xᵢ in M:
          aᵢ = argmin_j ||xᵢ - cⱼ||²

   c) UPDATE (using only mini-batch):
      For each cluster j:
          Compute per-cluster learning rate ηⱼ
          cⱼ = cⱼ + ηⱼ × (mean(M ∩ Cⱼ) - cⱼ)

      // Incremental update towards batch mean

   d) Check convergence (centroid movement < threshold)

3. Return final centroids
```

Key Differences:

1. **Per-iteration complexity:** O(b × k × d) instead of O(n × k × d)
2. Where b << n (e.g., b = 1000, n = 1,000,000)
3. **Speedup: ~n/b times faster per iteration**
4. **Incremental updates:** Centroids updated gradually using mini-batches
5. **Stochastic optimization:** More iterations needed, but each is much faster

Why It Works:

- **Convergence:** Mini-batch provides unbiased estimate of gradient
- **Net speedup:** Even with more iterations, total time is much less
- **Memory efficient:** Only b points in memory at once
- **Online learning:** Can handle streaming data

Practical Speedup:

For n = 10,000,000 users, k = 100 clusters, b = 10,000: - Standard k-means: ~10M distance calculations per iteration - Mini-batch: ~10K distance calculations per iteration - **1000× faster per iteration** - Even if needs 5× more iterations, still **200× faster overall**

Implementation:

```
from sklearn.cluster import MiniBatchKMeans
```

```
# Fast clustering for large dataset
kmeans = MiniBatchKMeans(
    n_clusters=100,
    batch_size=10000,      # Mini-batch size
    max_iter=300,
    random_state=42
)

kmeans.fit(large_feature_matrix)
```

Trade-offs:

- **Pros:** Much faster, scalable, memory efficient
- **Cons:** Slightly less accurate (typically 95-99% of standard k-means quality)
- **Acceptable:** For advertising use case, slight accuracy loss is worth massive speedup

Alternative Optimizations:

1. **Approximate nearest neighbors:** Use KD-trees or locality-sensitive hashing for faster assignment
2. **Parallel computing:** Distribute computation across multiple cores/machines
3. **Early stopping:** Stop when centroids barely move
4. **Smart initialization:** K-means++ reduces iterations needed

---

Part (b)(iv): Determining optimal k for Figure 2 [2 marks]

Visual Inspection of Figure 2:

Looking at the three clustering results (K=2, K=3, K=4):

Guess: K = 3 appears optimal

Reasoning: - K=2: The upper cluster should be split (clear gap between upper-left and upper-right groups) - K=3: Captures three natural groupings (two upper clusters, one lower cluster) - K=4: Over-segmentation - the upper-right cluster is artificially split

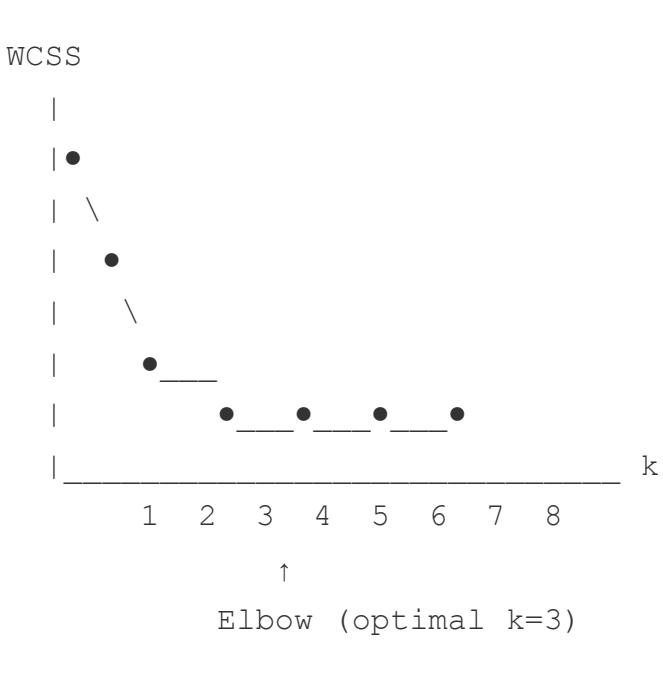
---

Method to Determine Correct Value of K:

Elbow Method (Primary Approach):

Process: 1. Run k-means for different values of k (e.g., k = 1 to 10) 2. For each k, compute the **Within-Cluster Sum of Squares (WCSS)**: $WCSS(k) = \sum_{j=1}^{k} \sum_{x_i \in C_j} ||x_i - c_j||^2$ 3. Plot WCSS vs. k 4. Look for the "elbow" - point where WCSS starts decreasing more slowly 5. The k value at the elbow is optimal

Elbow Plot:



Code:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

wcss = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)  # WCSS

plt.plot(K_range, wcss, 'bo-')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.show()
```

Alternative Methods:

1. Silhouette Score:

```
from sklearn.metrics import silhouette_score

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k)
    labels = kmeans.fit_predict(data)
    score = silhouette_score(data, labels)
    # Choose k with highest silhouette score
```

- Measures how similar points are to own cluster vs. other clusters
- Range: [-1, 1], higher is better
- Choose k that maximizes average silhouette score

2. Gap Statistic: - Compares WCSS of actual data vs. random data - Choose k where gap is largest

---

Single Run vs. Multiple Runs:

Question: Does it matter if k is determined over single run vs. many runs?

Answer: YES, multiple runs are important

Reasoning:

1. K-means is Non-deterministic: - Random initialization leads to different results - Can converge to different local minima - WCSS varies between runs

2. Impact on K Selection:

Single Run Problems: - May get unlucky initialization - WCSS might be artificially high or low - Elbow position can vary - **Example:** ``` Single run: k=3 gives WCSS = 150 (bad initialization) Single run: k=4 gives WCSS = 140 (good initialization) Conclusion: Wrongly choose k=4

Multiple runs average: k=3 average WCSS = 120 (true minimum) k=4 average WCSS = 140 (true minimum) Conclusion: Correctly choose k=3 ```

3. Best Practice - Multiple Runs:

```
def determine_optimal_k(data, k_range, n_runs=10):
    """Determine optimal k with multiple runs"""
    wcss_avg = []
```

```
    wcss_std = []

    for k in k_range:
        wcss_runs = []
        for run in range(n_runs):
            kmeans = KMeans(n_clusters=k, n_init=1, random_state=run)
            kmeans.fit(data)
            wcss_runs.append(kmeans.inertia_)

        wcss_avg.append(np.mean(wcss_runs))
        wcss_std.append(np.std(wcss_runs))

    # Plot average WCSS with error bars
    plt.errorbar(k_range, wcss_avg, yerr=wcss_std)
    plt.xlabel('k')
    plt.ylabel('Average WCSS')
    plt.show()

    return wcss_avg
```

4. Why Multiple Runs Matter:

- **Reduces variance:** Average over multiple initializations
- **More reliable:** Finds true pattern, not artifact of initialization
- **Confidence intervals:** Error bars show variability
- **Robust decision:** Based on typical performance, not lucky/unlucky run

5. Practical Recommendation:

- **Minimum:** 10 runs per k value
- **Better:** 50 runs for critical applications
- **Alternative:** Use k-means++ initialization (sklearn default: `n_init=10`)
- Better initialization reduces variability
- Still should do multiple runs for k selection

Sklearn Default:

```
# sklearn.cluster.KMeans default: n_init=10
kmeans = KMeans(n_clusters=3)  # Automatically runs 10 times, keeps b
```

Conclusion for Figure 2: - **Optimal k:** 3 (based on visual inspection and expected elbow method result) - **Methodology:** Run elbow method with 10+ runs per k value - **Report:** Average WCSS ± standard deviation for each k

---

Question 3: Database Systems (20 marks)

Part (a)(i): Compute blocking factor and number of blocks [2 marks]

Given Information:

- **Relation:** Employee(ID, Name, Age)
- **ID:** 64-bit integer = 8 bytes
- **Age:** 8-bit integer = 1 byte
- **Name:** 51 bytes
- **Number of tuples:** 1000
- **Block size:** 512 bytes
- **Block header:** 24 bytes
- **Record organization:** Fixed-length records

Step 1: Calculate Record Size

Each record (tuple) contains one ID, one Name, and one Age:

```
Record size R = sizeof(ID) + sizeof(Name) + sizeof(Age)
              = 8 bytes + 51 bytes + 1 byte
              = 60 bytes
```

Step 2: Calculate Available Space per Block

```
Available space per block = Block size - Header size
                          = 512 bytes - 24 bytes
                          = 488 bytes
```

Step 3: Calculate Blocking Factor (bfr)

The blocking factor is the number of records that fit in one block:

```
bfr = ⌊Available space / Record size⌋
    = ⌊488 / 60⌋
```

```
    = ⌊8.133...⌋
    = 8 records per block
```

Note: We use floor function because we can only fit complete records (fixed-length).

Step 4: Calculate Number of Blocks Required

Total number of blocks needed to store all 1000 tuples:

```
Number of blocks = ⌈Number of records / bfr⌉
                 = ⌈1000 / 8⌉
                 = ⌈125⌉
                 = 125 blocks
```

Verification: - 124 blocks hold: 124 × 8 = 992 records - 125th block holds: 1000 - 992 = 8 records - Total: 125 blocks needed

Answer: - **Blocking factor (bfr) = 8 records per block** - **Number of blocks required = 125 blocks**

Part (a)(ii): Query processing cost for different file organizations [5 marks]

Query:

```
SELECT Name FROM Employee WHERE ID >= 101 AND ID <= 115
```

Analysis: - Searching for records with ID in range [101, 115] - That's 15 consecutive IDs: 101, 102, 103, ..., 115 - Need to find these records and retrieve their Name attribute

Assumptions: - 1000 records total (ID from 1 to 1000, or similar distribution) - We'll assume IDs are somewhat uniformly distributed - Best case: all 15 records exist - Query retrieves 15 records

---

(a) Heap File Organization

Characteristics: - Records stored in no particular order - New records added at end of file - No index, no ordering

Search Strategy: - Must perform **linear scan** through entire file - Check each record to see if ID in range [101, 115] - Cannot stop early (records unordered)

Cost Analysis:

```
Cost = Number of blocks to scan
     = Total blocks in file
     = 125 blocks
```

Reasoning: - Must read every block to find all matching records - Matching records could be anywhere in the file - No way to skip blocks

Answer: 125 block accesses

(b) Sequential File (Ordered by Primary Key ID)

Characteristics: - Records physically ordered by ID - Can use binary search - Records with consecutive IDs likely in same or adjacent blocks

Search Strategy: 1. Use **binary search** to find first record (ID = 101) 2. Sequentially read records until ID > 115

Cost Analysis:

Step 1: Binary search for ID = 101

Binary search cost:

$$\text{Binary search cost} = \lceil \log_2(\text{number of blocks}) \rceil$$
$$= \lceil \log_2(125) \rceil$$
$$= \lceil 6.97 \rceil$$
$$= 7 \text{ block accesses}$$

Step 2: Sequential scan for range [101, 115]

How many blocks contain IDs 101-115?

With 1000 records in 125 blocks: - Average 8 records per block - 15 target records (ID 101-115) - If records clustered (ordered file), likely in: $\lceil 15/8 \rceil = 2$ blocks

However, we already accessed the first block during binary search.

Additional blocks to read:

```
Additional blocks = ⌈15 / 8⌉ - 1 = 2 - 1 = 1 block
```

Or more conservatively, assuming records span:

```
Additional blocks = ⌈15 / 8⌉ = 2 blocks
```

Total Cost:

```
Total cost = Binary search + Sequential scan
           = 7 + 2
           = 9 block accesses
```

Alternative calculation: If we assume the 15 records are contiguous: - First record (ID=101) found with binary search: 7 accesses - Reading range [101-115]: These fit in ~2 blocks - Total: 7 + 2 = 9 blocks

Answer: 9 block accesses

(c) Hash File (External Hashing)

Characteristics: - Hash function maps ID to bucket - 256 buckets, each containing 1 block - No overflow buckets - Hash key field: ID (primary key)

Search Strategy: - For each ID in range [101, 115], compute hash and access corresponding bucket - Each ID hashes to potentially different bucket

Cost Analysis:

Assumptions about hash function: - Good hash function distributes records uniformly - Each ID hashes to a random bucket (uniform distribution) - IDs 101-115 likely hash to different buckets (sequential IDs → random buckets)

Searching for range [101, 115]:

```
For each ID from 101 to 115:
    bucket = hash(ID)
```

```
    Access block at bucket[ID]
    Check if record exists
```

Number of IDs to check: 15

Expected accesses: - In the worst case (different buckets): 15 block accesses - In best case (same bucket): 1 block access - **Expected:** With good hash function, most IDs map to different buckets

Realistic estimate: 15 block accesses

Alternative consideration: If hash function is h(ID) = ID mod 256, then: - hash(101) = 101 - hash(102) = 102 - ... - hash(115) = 115

All hash to different buckets → 15 different block accesses

Answer: 15 block accesses

Summary Comparison:

File Organization	Cost (Block Accesses)	Method
(a) Heap File	**125**	Linear scan of entire file
(b) Sequential File	**9**	Binary search (7) + range scan (2)
(c) Hash File	**15**	One access per ID in range

Conclusion: - **Best:** Sequential file (9 accesses) - benefits from ordering for range queries - **Worst:** Heap file (125 accesses) - must scan everything - **Middle:** Hash file (15 accesses) - good for point queries, poor for range queries

Key Insights: - **Range queries** favor ordered (sequential) files - **Point queries** (single ID) favor hash files - **Heap files** are worst for selective queries (good only for full table scans)

Part (b)(i): Nested-loop join strategies and costs [6 marks]

Given Information:

Relation E (Employee): - n_e = 100 blocks - r_e = 1000 records

Relation D (Department): - n_D = 50 blocks - r_D = 10 records

Other parameters: - Memory buffer: n_B = 12 blocks - Blocking factor for result: bfr_{rs} = 10 records per block - Join condition: D.Mgr_SSN = E.SSN (equi-join) - Mgr_SSN is unique (1-to-1 relationship)

Query:

```
SELECT * FROM E, D WHERE D.Mgr_SSN = E.SSN
```

Nested-Loop Join Algorithm (Basic):

```
For each block of outer relation:
    Load outer block into memory
    For each block of inner relation:
        Load inner block into memory
        For each tuple in outer block:
            For each tuple in inner block:
                If join condition matches:
                    Output joined tuple
```

Strategy 1: E as outer, D as inner (E ⋈ D)

Configuration: - Outer relation: E (100 blocks) - Inner relation: D (50 blocks) - Memory allocation: - 1 block for outer relation (E) - 1 block for inner relation (D) - Remaining blocks for buffers

Algorithm:

```
For each block b_E in E (100 blocks):
    Read b_E into memory                    [1 access]
    For each block b_D in D (50 blocks):
        Read b_D into memory                [1 access]
        Perform join on tuples in b_E and b_D
```

Cost Calculation:

```
Cost = (Read E) + (Read D for each block of E)
     = n_e + (n_e × n_D)
     = 100 + (100 × 50)
     = 100 + 5000
     = 5100 block accesses
```

Breakdown: - Read E once: 100 blocks - For each of 100 E blocks, read all of D: 100 × 50 = 5000 blocks - Total: 5100 blocks

---

Strategy 2: D as outer, E as inner (D ⋈ E)

Configuration: - Outer relation: D (50 blocks) - Inner relation: E (100 blocks)

Algorithm:

```
For each block b_D in D (50 blocks):
    Read b_D into memory                    [1 access]
    For each block b_E in E (100 blocks):
        Read b_E into memory                [1 access]
        Perform join on tuples in b_D and b_E
```

Cost Calculation:

```
Cost = (Read D) + (Read E for each block of D)
     = n_D + (n_D × n_e)
     = 50 + (50 × 100)
     = 50 + 5000
     = 5050 block accesses
```

Breakdown: - Read D once: 50 blocks - For each of 50 D blocks, read all of E: 50 × 100 = 5000 blocks - Total: 5050 blocks

---

Strategy 3: Block Nested-Loop Join with D as outer

Configuration: - Use available memory buffer n_B = 12 blocks - Outer relation: D (50 blocks) - Inner relation: E (100 blocks) - Memory allocation: - n_B - 2 = 10 blocks for outer relation D - 1 block for inner relation E - 1 block for output buffer

Algorithm:

```
For each chunk of (n_B - 2) blocks of D:
    Read n_B - 2 blocks of D into memory
    For each block b_E in E:
        Read b_E into memory
        Perform join on all tuples
```

Number of chunks of D:

```
Chunks of D = ⌈n_D / (n_B - 2)⌉
            = ⌈50 / 10⌉
            = 5 chunks
```

Cost Calculation:

```
Cost = (Read D) + (Chunks of D × Read E)
     = n_D + (⌈n_D / (n_B - 2)⌉ × n_e)
     = 50 + (5 × 100)
     = 50 + 500
     = 550 block accesses
```

Breakdown: - Read D once: 50 blocks - For each of 5 chunks of D, read all of E: 5 × 100 = 500 blocks - Total: 550 blocks

---

Strategy 4: Block Nested-Loop Join with E as outer

Configuration: - Memory buffer n_B = 12 blocks - Outer relation: E (100 blocks) - Inner relation: D (50 blocks) - Memory allocation: - n_B - 2 = 10 blocks for outer relation E - 1 block for inner relation D - 1 block for output buffer

Number of chunks of E:

```
Chunks of E = ⌈n_e / (n_B - 2)⌉
            = ⌈100 / 10⌉
            = 10 chunks
```

Cost Calculation:

```
Cost = (Read E) + (Chunks of E × Read D)
     = n_e + (⌈n_e / (n_B - 2)⌉ × n_D)
     = 100 + (10 × 50)
     = 100 + 500
     = 600 block accesses
```

Breakdown: - Read E once: 100 blocks - For each of 10 chunks of E, read all of D: 10 × 50 = 500 blocks - Total: 600 blocks

---

Summary and Conclusion:

Strategy	Outer	Inner	Cost Formula	Cost (blocks)
1. Simple NL	E	D	$n_e + (n_e \times n\_D)$	5100
2. Simple NL	D	E	$n\_D + (n\_D \times n_e)$	5050
3. Block NL	D	E	$n\_D + \lceil n\_D/(n\_B-2) \rceil \times n_e$	550
4. Block NL	E	D	$n_e + \lceil n_e/(n\_B-2) \rceil \times n\_D$	600

Most Efficient Strategy: Strategy 3 (Block Nested-Loop with D as outer)

Cost: 550 block accesses

Why this is best: 1. **Uses memory efficiently:** Loads 10 blocks of D at once 2. **Smaller outer relation:** D has fewer blocks (50 vs 100) 3. **Minimizes inner relation reads:** Only reads E 5 times instead of 50 times

General Principle: For block nested-loop join, choose the **smaller relation as outer** to minimize the number of times the inner relation is scanned.

```
Optimal: Smaller relation as outer, larger as inner
Cost = n_smaller + ⌈n_smaller/(n_B-2)⌉ × n_larger
```

Part (b)(ii): Index-based nested-loop join strategies [7 marks]

Given Indexes: - **Level 2 Secondary Index** on D.Mgr_SSN (unique attribute) - **Level 2 Primary Index** on E.SSN (primary key)

Index Structure Reminder:

Multi-level Index: - **Level 1:** Index entries pointing to data blocks - **Level 2:** Index on the Level 1 index (index of index) - **Level x:** Number of levels in the index hierarchy

Access cost: - **Level x index access:** x + 1 block accesses - x blocks to navigate index levels - 1 block to access data

---

Strategy 1: Iterate over D, use index on E

Algorithm:

```
For each tuple d in D:
    mgr_ssn = d.Mgr_SSN
    Use primary index on E to find matching record with E.SSN = mgr_s
    Output joined tuple
```

Cost Calculation:

Step 1: Read all blocks of D

```
Cost to read D = n_D = 50 blocks
```

Step 2: For each tuple in D, use primary index on E

Number of tuples in D: r_D = 10

For each tuple in D: - Use Level 2 Primary Index on E.SSN - Cost per index lookup = x_e + 1 = 2 + 1 = 3 block accesses - 2 blocks to navigate index levels - 1 block to access data record

Total index access cost:

```
Index cost = r_D × (x_e + 1)
           = 10 × 3
           = 30 block accesses
```

Total Cost:

```
Total cost = n_D + (r_D × (x_e + 1))
           = 50 + 30
           = 80 block accesses
```

Strategy 2: Iterate over E, use index on D

Algorithm:

```
For each tuple e in E:
    ssn = e.SSN
    Use secondary index on D to find matching record with D.Mgr_SSN =
    If match found:
        Output joined tuple
```

Cost Calculation:

Step 1: Read all blocks of E

```
Cost to read E = n_e = 100 blocks
```

Step 2: For each tuple in E, use secondary index on D

Number of tuples in E: $r_e = 1000$

For each tuple in E: - Use Level 2 Secondary Index on D.Mgr_SSN - Cost per index lookup = x_D + 1 = 2 + 1 = 3 block accesses

Total index access cost:

```
Index cost = re × (x_D + 1)
           = 1000 × 3
           = 3000 block accesses
```

Total Cost:

```
Total cost = ne + (re × (x_D + 1))
           = 100 + 3000
           = 3100 block accesses
```

Comparison and Conclusion:

Strategy	Outer	Index Used	Cost Formula	Cost (blocks)
1	D (10 records)	Primary index on E	$n_D + r_D \times (x_e+1)$	80
2	E (1000 records)	Secondary index on D	$n_e + r_e \times (x_D+1)$	3100

Best Strategy: Strategy 1

Cost: 80 block accesses

Why Strategy 1 is Better:

1. **Fewer index lookups:**
2. Strategy 1: 10 index lookups (one per department)
3. Strategy 2: 1000 index lookups (one per employee)
4. **Smaller outer relation:**
5. D has only 10 records
6. E has 1000 records
7. Index lookup cost is O(number of outer tuples)

8. **Efficient use of primary index:**
9. Primary index on E is clustered (records ordered by SSN)
10. Fast access to specific SSN value
11. **Cardinality consideration:**
12. Since Mgr_SSN is unique and references E.SSN
13. Only 10 employees are managers (match)
14. Strategy 1 does 10 lookups (exactly the matches)
15. Strategy 2 does 1000 lookups (990 fail, 10 succeed)

General Principle for Index-Based Nested-Loop Join:

```
Choose the smaller relation as outer relation
Use index on the larger relation (inner)

Cost = n_outer + (r_outer × (index_level + 1))
```

Optimization tip: - If one relation is much smaller, always make it outer - Use index on the larger relation (inner) - Especially effective when: - Outer relation is small - Index on inner relation is efficient (few levels) - Join is selective (few matches)

Summary of All Join Strategies (Question 3b):

Method	Strategy	Cost	Rank
Index NL	D outer, index on E	80	1st (BEST)
Block NL	D outer, E inner	550	2nd
Block NL	E outer, D inner	600	3rd
Index NL	E outer, index on D	3100	4th
Simple NL	D outer, E inner	5050	5th

Method	Strategy	Cost	Rank
Simple NL	E outer, D inner	5100	6th (WORST)

Recommendation: Use Index-based Nested-Loop Join with D as outer relation (80 block accesses)

End of Solutions

Summary: - **Question 1 (20 marks):** Linear algebra, probability, visualisation and optimisation - **Question 2 (20 marks):** Text processing in data science - **Question 3 (20 marks):** Database systems - **Total: 60 marks**

Key Topics Covered: - Matrix operations and gradient descent - Multivariate normal distributions and eigendecomposition - Text processing (TF-IDF, cosine similarity, lemmatization, k-means clustering) - Database query optimization and join strategies

---

Prepared by: AI Assistant Date: November 25, 2025 For: IDSS 2019-2020 Examination